Distributed Analysis of US Airline Delays
using Apache Spark and HDFS
SABD 2025/26 — Project 1

Daniel Garoz

M.Sc. Computer Engineering — Erasmus Exchange
Università degli Studi di Roma "Tor Vergata"

June 12, 2026

Outline

Project Goal

What the project asks

Build a Big Data pipeline that uses **Apache Spark** to answer analytical queries over US flight data, with the dataset stored in **HDFS** and the results written back to HDFS.

Why this matters (the real evaluation)

The goal is **not** to learn about airlines; the airline data is just the test bench. What is evaluated is:

- Architecture and deployment (HDFS + Spark + Docker)
- Code organization (separation of concerns)
- Efficiency (cache, shuffle, coalesce decisions)
- Performance evaluation (proper benchmarking)

Dataset — US BTS On-Time Performance

Source: US Bureau of Transportation Statistics
Period available: January–March 2025
Total flights: 1,554,583

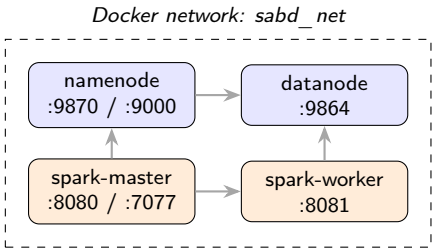
27 original columns → project only the **14 needed** for Q1+Q2:

- Temporal: `YEAR`, `MONTH`, `DAY_OF_MONTH`, `CRS_DEP_TIME`
- Carrier: `OP_UNIQUE_CARRIER`
- Delays: `DEP_DELAY`, `ARR_DELAY`
- Status: `CANCELLED`, `DIVERTED`
- 5 delay causes: `CARRIER`, `WEATHER`, `NAS`, `SECURITY`, `LATE_AIRCRAFT`

Month	Records
January	539,747
February	504,884
March	604,000
April	583,950
Total	**≈2.23 M**

Dataset covers the full Jan–Apr 2025 window (≈2.23 M flights).

System Architecture — Containerized Cluster



HDFS layer (storage)

- `bde2020/hadoop-namenode`
- `bde2020/hadoop-datanode`
- Replication 1 (single-node teaching)

Spark layer (compute)

- `bde2020/spark-master`
- `bde2020/spark-worker`
- Spark 3.3.0

Orchestration: Docker Compose, one YAML, reproducible.

Bring it up: `docker compose -f docker/docker-compose.yml up -d`

Code Organization

Three layers mirror the runtime:

Reusable helpers — `utils.py`, `preprocessing.py`

- `utils.py`: timing context manager, list of relevant columns
- `preprocessing.py`: SparkSession construction, schema-aware loader

One script per query

- `query1.py`, `query2.py`: pure query logic
- `plot_q1.py`, `plot_q2.py`: visualization (pandas + matplotlib)
- `benchmark.py`: harness that runs queries N times for stats

Migration local → HDFS was 1 line of code

Only `preprocessing.py` had to read the Spark master URL from an environment variable. Everything else is unchanged.

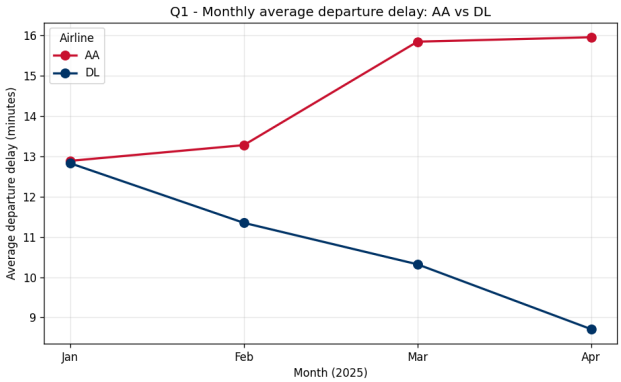
Query 1 — Monthly stats for AA and DL

Goal: for American Airlines and Delta, per month:

- average / minimum / maximum `DEP_DELAY` (non-cancelled flights only)
- cancellation rate (over all flights)

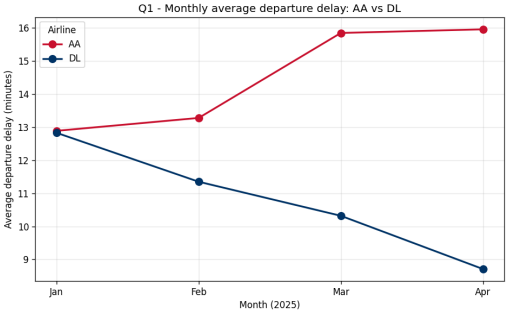
Implementation key points

- Filter to {AA, DL}
- `cache()` the filtered DataFrame
- It feeds **two** aggregations:
  - delay stats (non-cancelled subset)
  - cancellation rate (all)
- `join` on (month, airline)
- `coalesce(1)` for single CSV

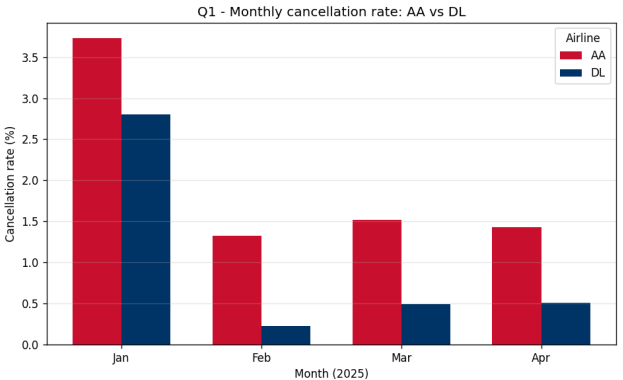


Monthly avg departure delay (AA vs DL)

Query 1 — Results



Average departure delay



Cancellation rate

What we observe:

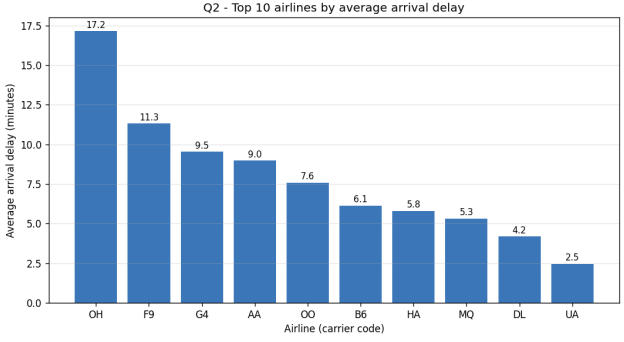
- Average delay is comparable between AA and DL (∼8–15 min)
- Delta has a **much lower cancellation rate** (peak 2.8% in Jan vs AA peak 3.7%)
- January is the worst month for both (winter weather)

Query 2 — Top 10 by arrival delay

Goal: rank carriers by mean `ARR_DELAY`, considering only carriers with at least 500 *completed* flights, and break down the contribution of the 5 delay causes.

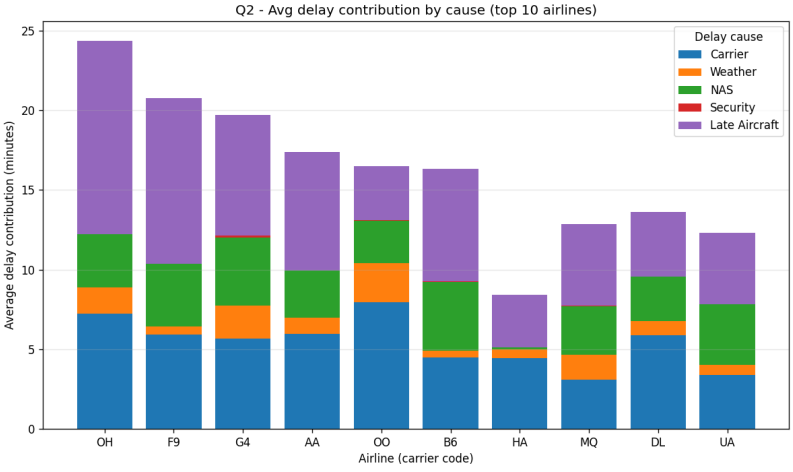
Implementation key points

- Filter: `CANCELLED=0 AND DIVERTED=0`
- NULL delay-cause columns $\rightarrow$ 0 (BTS convention: filled only when ARR_DELAY $\geq$ 15)
- Single `groupBy` with 7 aggregations in parallel
- Filter `num_flights` $\geq$ 500
- `orderBy desc + limit(10)`



Top 10 carriers ranked by mean arrival delay

Query 2 — Cause breakdown



Dominant pattern: `LATE_AIRCRAFT_DELAY` is the largest contributor for the worst-performing carriers (OH, F9).

This is the **cascading effect**: one delay propagates downstream through the day's flight rotation

Four design decisions I would defend

1. Explicit schema (`StructType`) instead of `inferSchema`

`inferSchema=true` would re-scan the entire CSV just to deduce types. Explicit schema halves loading time and prevents silent type mismatches.

2. `cache()` only when the DataFrame is reused

Q1 caches (used twice). Q2 does not (used once). Caching for the sake of caching wastes memory and triggers an extra materialization.

3. `shuffle.partitions = 8` (Spark default: 200)

For 1.5 M records on one machine, 200 partitions create scheduling overhead with mostly-empty work units. Setting it to the CPU count removes that overhead.

4. `coalesce(1)` at output — a trade-off

Forces a final shuffle but produces **a single CSV per query** as required by the spec. Removing it would save $\sim$2 s but force the user to concatenate eight part-files by hand.

Benchmark methodology

Two configurations:

- **Local**: PySpark in venv, `master("local[*]")`, CSVs on disk
- **HDFS**: `spark-submit` on the dockerized cluster, CSVs in HDFS

Protocol per query, per configuration: 6 runs = 1 warm-up (discarded) + 5 measured

Why warm-up matters: pilot run gave Q1 end-to-end of **57.2 s** on HDFS, against the steady-state **22.1 s**. The 35-second gap is JVM JIT compilation + HDFS metadata caching.

Reported: mean $\pm$ sample standard deviation across the 5 measured runs.

Results — Local vs HDFS

Stage	Local		HDFS cluster	
	Q1	Q2	Q1	Q2
loading	5.41 ± 0.90	1.26 ± 0.73	18.20 ± 1.35	12.26 ± 1.40
preprocessing	0.02 ± 0.00	0.12 ± 0.03	0.02 ± 0.00	0.14 ± 0.06
computation	0.17 ± 0.02	0.17 ± 0.08	0.22 ± 0.04	0.14 ± 0.01
output	1.80 ± 0.33	4.94 ± 1.54	3.68 ± 0.43	6.60 ± 0.16
end-to-end	7.40 ± 1.17	6.49 ± 2.36	22.12 ± 1.72	19.15 ± 1.54

Overhead: $3.0\times$ for Q1, $2.95\times$ for Q2 when moving to HDFS.

Observations

1. Loading dominates the HDFS overhead

3.4× slower for Q1, 9.7× for Q2. Cost of: namenode RPC, block-location lookup, streaming from datanode.

2. Output is the second-largest HDFS cost (Q2: 6.6 s)

`coalesce(1)` shuffle + HDFS write path (block write, metadata update, replication ack), amplified by container network.

3. Computation is essentially unchanged

Data volume is small relative to single-CPU capacity. The cluster adds *infrastructure* cost without adding *compute* cost at this scale.

4. Steady-state $\neq$ first run

Warm-up absorbs 5–15× the steady-state cost. Mandatory in any honest benchmark.

Conclusions

Delivered

- Containerized HDFS + Spark cluster (1 YAML)
- Q1 and Q2 implemented with the Spark DataFrame API
- 4 plots, 2 CSV deliverables, full benchmark suite
- Local vs HDFS comparison, end-to-end and per stage

Main take-aways

- HDFS overhead is real, measurable and predictable at this scale
- Most performance wins come from *decisions*, not from *frameworks* (cache, schema, partitioning, coalesce)
- Reproducibility is achievable: one `compose up` brings the cluster, two scripts complete the pipeline

Thank you

Questions?

Daniel Garoz – SABD 2025/26 – Project 1